

TP 1: Mesure expérimentale du coût d'algorithmes élémentaires

BUT Informatique - S5 - Qualité algorithmique

1 Introduction

L'objectif de ce TP est d'estimer expérimentalement le coût de différents algorithmes dans le pire cas, le meilleur cas et en moyenne.

1.1 Implantation de quelques algorithmes simples

Implémentez en méthodes Java les algorithmes 3, 4 et 6 de l'énoncé de TD 1, ainsi que l'algorithme de recherche linéaire suivant:

```
function Recherche-linéaire(tab,val)
  i <- 0
  while i < longueur(tab) and tab[i] != val do
    i++
  end
  if (i<longueur(tab))
    return i
  else
    return -1
  end
end
```

1.2 Ajout de mesures de coût

Dans la suite, nous allons utiliser deux mesures de coût:

- la première en chronométrant l'exécution des fonctions à l'aide de la méthode `System.nanoTime()`
- la seconde en comptant le nombre d'opérations élémentaires à l'aide de ce

compteur:

```
static class OperationCounter {  
    private long count = 0;  
    public void tick() { count++; }  
    public long get() { return count; }  
    public void reset() { count = 0; }  
}
```

Le chronomètre ne fournit généralement pas une mesure correcte du coût, car il tient compte du “bruit” de l’environnement d’exécution (JIT compilation, Garbage Collector de la JVM, interruptions système). Cependant, il peut être intéressant d’évaluer sa qualité pour mesurer le coût des algorithmes.

Avec le compteur d’opérations on fera abstraction du temps réellement consommé par les opérations élémentaires pour ne comptabiliser que leur nombre.

Ajoutez en paramètre de vos fonctions un compteur d’opérations et utilisez-le dans le corps de vos fonctions pour réaliser le comptage des opérations.

2 Protocole Expérimental

Pour les expérimentations, vous utiliserez la fonction suivante de génération aléatoire de tableaux:

```
import java.util.Random;  
static final Random RANDOM = new Random(42);  
static final int BOUND = 1_000_000;  
  
static int[] randomArray(int size, int bound) {  
    int[] array = new int[size];  
    for (int i = 0; i < size; i++) {  
        array[i] = RANDOM.nextInt(bound);  
    }  
    return array;  
}
```

Une expérience consiste à mesurer le temps et le nombre d’opérations réalisées par un algorithme pour une longueur `N` de tableau donné en entrée (et une valeur à rechercher donnée en entrée pour l’algorithme de recherche).

Pour obtenir des mesures temporelles fiables (`System.nanoTime()`), vous devez :

1. **Chauffer la JVM (Warm-up)** : Exécuter chaque fonction 10 000 fois sur des petits tableaux avant de démarrer vos mesures.
2. **Répéter les mesures** : Pour une taille `N` donnée, exécuter l’expérience 30 fois et calculer la **moyenne** des temps.

2.1 Évaluation du coût dans chaque cas (Pire, Meilleur, Moyen)

Pour chaque algorithme, faites varier la taille de 100 à 1 000 000 (en adaptant la borne max pour l'algorithme quadratique afin d'éviter que le TP ne dure des heures!).

Pour configurer vos tests, ces questions préalables pourraient vous aider:

- **Algorithmes 3, 4 et 6:** Le contenu du tableau influence-t-il le nombre d'opérations ? Justifiez.
- **Recherche Linéaire :**
 - **Meilleur et pire cas :** Quelles valeurs rechercher ?
 - **Cas moyen :** Cherchez une valeur présente à un index aléatoire. Répétez l'opération 50 fois pour en faire la moyenne.

2.2 Analyse graphique

Reportez les valeurs trouvées sur un graphe permettant de comparer les coûts des différents algorithmes et constater qu'ils correspondent à ce qui a été trouvé en TD.

Reportez vos résultats sur deux graphiques distincts (un pour le compteur d'opérations, un pour le temps en nanosecondes).

- Utilisez une **échelle logarithmique (Log-Log)**.
- Identifiez visuellement les différentes complexités $O(1)$, $O(\log n)$, $O(n)$ et $O(n^2)$.
- Les courbes du compteur d'opérations et du temps ont-elles la même allure ? Concluez sur la pertinence de chaque méthode de mesure.

Aide: Export des données au format CSV avec `PrintWriter` et affichage des graphes

Pour l'affichage vous pourrez utiliser le script Python disponible sur Moodle. Celui-ci nécessite deux fichiers au format CSV qui devront être produits par votre programme Java:

- `algos_generaux.csv` doit contenir les résultats des expériences pour les algorithmes du TD. La première ligne du fichier (l'en-tête) doit définir le nom des colonnes, et chaque ligne suivante représentera les résultats pour une taille `N` donnée:
`N, op_algo3, t_algo3, op_algo4, t_algo4, op_algo6, t_algo6`).

- `recherche_lineaire.csv` doit contenir les résultats des expériences pour la recherche linéaire sur le même modèle que le fichier précédent mais avec ces colonnes:
`N, op_best, t_best, op_worst, t_worst, op_avg, t_avg`.

⚠ Attention au piège de la complexité quadratique ($O(n^2)$): Comme vous le constaterez vite, l'algorithme 4 devient extrêmement long pour les grandes valeurs de `N` (au-delà de `N = 10 000`). Pour éviter que votre programme Java ne gèle pendant des heures, vous devez brider son exécution à `N ≤ 10000`. Pour que votre fichier CSV reste valide et lisible par la bibliothèque Python Pandas, les lignes correspondant à des valeurs de `N > 10 000` ne doivent pas contenir de "0" artificiel (ce qui fausserait vos graphiques), mais rester vides. Nous vous conseillons donc d'utiliser des variables de type String pour l'algorithme 4:

- Si `N ≤ 10 000`, convertissez vos mesures numériques en texte avec `String.valueOf(mesure)`.
- Si `N > 10 000`, laissez ces variables à l'état de chaîne vide `""`.

En utilisant un `PrintWriter` combiné à la méthode `pw.printf("%d,%d,%d,%s,%s,%d,%d\n", ...)`, les chaînes vides généreront automatiquement des virgules successives (ex: 100000,400003,154200,,,34,120).

3 Un peu plus loin sur le cas moyen

Dans la section 3, pour mesurer le cas moyen, nous avons choisi une valeur déjà présente à un index aléatoire du tableau. Nous allons maintenant modifier la méthode de sélection de la valeur cible.

Modifiez temporairement votre code Java pour que la valeur recherchée soit générée par `RANDOM.nextInt(bound)` (comme pour le remplissage du tableau). Faites tourner le test pour une taille fixe (`N = 10 000`), en faisant varier la borne supérieure (`bound`) : essayez successivement 1000, 5000, 20000, 100 000, 1 000 000. Affichez le nombre moyen d'opérations pour chaque configuration.

- Qu'observez-vous sur le nombre d'opérations moyennes lorsque la valeur de `bound` augmente, alors que la taille du tableau `N` reste inchangée ? Vers quelle complexité théorique connue le cas moyen semble-t-il converger lorsque `bound` devient très grand devant `N` ?
- Quelle est la probabilité théorique qu'un entier généré aléatoirement entre 0 et `bound - 1` soit présent dans un tableau de taille `N` (en supposant, pour simplifier, que les éléments du tableau sont tous distincts) ? Que devient cette probabilité lorsque `bound` tend vers l'infini ?

- Expliquez pourquoi le choix de la méthode de génération de la cible (valeur issue du tableau vs valeur issue de $0.. \text{bound} - 1$) modifie si radicalement le comportement de l'algorithme.
- En informatique, la notion de "complexité en moyenne" nécessite toujours de définir une hypothèse sur la distribution statistique des entrées. Au vu de vos résultats, pourquoi est-il dangereux de tester le "cas moyen" d'un algorithme de recherche en choisissant des données purement aléatoires sans analyser le profil des données réelles de l'application ?

C. Lhoussaine